

```
import paramiko
import os
import time
import msvcrt # Teclado para Windows

# Importações da biblioteca Rich
from rich.console import Console
from rich.panel import Panel
from rich.table import Table
from rich.layout import Layout
from rich.text import Text
from rich.live import Live
from rich.columns import Columns

# =====
# CONFIGURAÇÕES DE ACESSO AO SERVIDOR LINUX (ASH/BUSYBOX)
# =====
SSH_HOST = "172.16.35.1"
SSH_USER = "root"
SSH_PASS = "h4rdFlip"

# Configuração de escala para o gráfico de rede (Ex: 10 MB/s equivale a 100% da barra)
REDE_MAX_VELOCIDADE_MBS = 10.0

console = Console()
ULTIMO_ERRO = ""

# Variáveis globais para calcular a velocidade da rede
g_bytes_in_ant = 0.0
g_bytes_out_ant = 0.0
g_tempo_ant = 0.0

def obter_metricas_servidor(ssh):
    """Executa e extrai todas as métricas cruas de forma isolada e segura"""
    global ULTIMO_ERRO
    try:
        # 1. Captura estritamente a linha GLOBAL da cpu (ignora cpu0, cpu1, etc.)
        stdin, stdout, stderr = ssh.exec_command("cat /proc/stat | grep '^cpu '")
        cpu1_linha = stdout.readline().strip().split()

        time.sleep(0.5) # Janela de amostragem para variação de ciclos

        # 2. Captura o segundo estado da CPU de forma isolada
        stdin, stdout, stderr = ssh.exec_command("cat /proc/stat | grep '^cpu '")
        cpu2_linha = stdout.readline().strip().split()

        # 3. Executa as demais métricas sem misturar com o arquivo /proc/stat
        comando_demais = (
            "cat /proc/sys/kernel/hostname ; "
            "cat /proc/meminfo | grep -E 'MemTotal|MemAvailable|MemFree' | awk '{print $2}' | "
            tr '\\n' '|' ; echo ; "
            "df -h / | awk 'NR==2 {print $5}' || df -h / | awk 'NR==3 {print $4}' ; "
            "interface=$(ip route | grep default | awk '{print $5}'); "
            "cat /proc/net/dev | grep \"$interface\" | awk '{print $2\"|\"$10}'"
        )

        stdin, stdout, stderr = ssh.exec_command(comando_demais)
        linhas_bloco = [l.strip() for item in stdout.readlines() if (l := item.strip())]

        if len(linhas_bloco) < 4:
            ULTIMO_ERRO = "Resposta incompleta do servidor nas métricas de hardware."
            return None
```

```
# --- Processamento de CPU ---
uso_cpu = 0.0
if cpu1_linha and cpu2_linha and cpu1_linha[0] == 'cpu' and cpu2_linha[0] == 'cpu':
    c1_v = [int(x) for x in cpu1_linha[1:]]
    c2_v = [int(x) for x in cpu2_linha[1:]]
    diff_total = sum(c2_v) - sum(c1_v)
    diff_idle = c2_v[3] - c1_v[3] # Índice 3 é o tempo ocioso (idle)
    if diff_total > 0:
        uso_cpu = round((1.0 - (diff_idle / diff_total)) * 100, 2)

# --- Processamento Seguro das Demais Linhas (Garantindo os Índices Fixos) ---
hostname = linhas_bloco[0]

# Memória (Linha 1)
dados_mem = linhas_bloco[1].split('|')
mem_total = float(dados_mem[0])
# Se o sistema compactado não tiver MemAvailable (índice 1 vazio), usa MemFree
(índice 2)
mem_livre = float(dados_mem[1]) if len(dados_mem) > 2 and dados_mem[1] else
float(dados_mem[2])
perc_mem = round(((mem_total - mem_livre) / mem_total) * 100, 2)
tam_memoria = f"{mem_total / (1024**2):.2f} GB"

# Disco (Linha 2)
perc_disco = float(linhas_bloco[2].replace('%', ''))

# Rede (Linha 3)
bytes_in_raw, bytes_out_raw = linhas_bloco[3].split('|')

return {
    "hostname": hostname,
    "uso_cpu": uso_cpu,
    "uso_memoria": perc_mem,
    "tam_memoria": tam_memoria,
    "perc_disco": perc_disco,
    "bytes_in": float(bytes_in_raw),
    "bytes_out": float(bytes_out_raw)
}
except Exception as e:
    ULTIMO_ERRO = f"Erro na coleta: {e}"
    return None

def coletar_dados_com_velocidade():
    global g_bytes_in_ant, g_bytes_out_ant, g_tempo_ant, ULTIMO_ERRO

    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    try:
        ssh.connect(SSH_HOST, username=SSH_USER, password=SSH_PASS, timeout=3)
        res = obter_metricas_servidor(ssh)

        if not res:
            return None

        tempo_atual = time.time()
        vel_in_mbs = 0.0
        vel_out_mbs = 0.0

        if g_tempo_ant > 0:
            diff_tempo = tempo_atual - g_tempo_ant
```

```
        if diff_tempo > 0:
            vel_in_mbs = ((res["bytes_in"] - g_bytes_in_ant) / (1024**2)) / diff_tempo
            vel_out_mbs = ((res["bytes_out"] - g_bytes_out_ant) / (1024**2)) / diff_tempo

            vel_in_mbs = max(0.0, vel_in_mbs)
            vel_out_mbs = max(0.0, vel_out_mbs)

        g_bytes_in_ant = res["bytes_in"]
        g_bytes_out_ant = res["bytes_out"]
        g_tempo_ant = tempo_atual

        perc_grafico_in = min(100.0, (vel_in_mbs / REDE_MAX_VELOCIDADE_MBS) * 100)
        perc_grafico_out = min(100.0, (vel_out_mbs / REDE_MAX_VELOCIDADE_MBS) * 100)

        return {
            "IP": SSH_HOST,
            "HOSTNAME": res["hostname"],
            "USO_CPU": res["uso_cpu"],
            "USO_MEMORIA": res["uso_memoria"],
            "TAMANHO_MEMORIA": res["tam_memoria"],
            "DISCO_ROOT_PERC": res["perc_disco"],
            "REDE_IN_MBS": round(vel_in_mbs, 2),
            "REDE_OUT_MBS": round(vel_out_mbs, 2),
            "PERC_GRAFICO_IN": round(perc_grafico_in, 1),
            "PERC_GRAFICO_OUT": round(perc_grafico_out, 1),
            "REDE_TOTAL_IN_MB": round(res["bytes_in"] / (1024**2), 1),
            "REDE_TOTAL_OUT_MB": round(res["bytes_out"] / (1024**2), 1)
        }
    except Exception as e:
        ULTIMO_ERRO = f"Falha de conexão: {e}"
        return None
    finally:
        ssh.close()

def desenhar_barra(percentagem, texto_sufixo, cor):
    largura_maxima = 18
    blocos_preenchidos = int((percentagem / 100) * largura_maxima)
    blocos_vazios = largura_maxima - blocos_preenchidos

    barra = f"[{cor}]" + "█" * blocos_preenchidos + "[/]"
    barra += "[grey37]" + "░" * blocos_vazios + "[/]"
    return f"[{barra}] {texto_sufixo}"

def gerar_layout_dashboard(dados):
    header_text = Text("📊 DASHBOARD INTEGRADO DE TELEMETRIA (REDE E HARDWARE)", style="bold
white on navy_blue", justify="center")
    header_panel = Panel(header_text, style="blue", padding=(0, 1))

    if not dados:
        erro_formatado = f"\n⚠️ Sincronizando dados com o servidor remoto...\n\n[bold
yellow>Status:[/bold yellow] {ULTIMO_ERRO}\n"
        return Panel(Text(erro_formatado, style="bold red", justify="center"), title="Status
do Sistema", border_style="red")

    cor_cpu = "green" if dados["USO_CPU"] < 50 else "yellow" if dados["USO_CPU"] < 80 else
"red"
    cor_mem = "green" if dados["USO_MEMORIA"] < 70 else "yellow" if dados["USO_MEMORIA"] < 90
else "red"
    cor_disco = "green" if dados["DISCO_ROOT_PERC"] < 75 else "yellow" if
dados["DISCO_ROOT_PERC"] < 90 else "red"

    # --- COLUNA DA ESQUERDA: HARDWARE ---
```

```

    tab_hardware = Table(show_header=False, expand=True, box=None)
    tab_hardware.add_row("[bold white]Uso de CPU[/bold white]")
    tab_hardware.add_row(desenhar_barra(dados["USO_CPU"], f"[bold {cor_cpu}]
{dados['USO_CPU']}%[/]", cor_cpu))
    tab_hardware.add_row("")
    tab_hardware.add_row("[bold white]Uso de Memória RAM[/bold white]")
    tab_hardware.add_row(desenhar_barra(dados["USO_MEMORIA"], f"[bold {cor_mem}]
{dados['USO_MEMORIA']}%[/]", cor_mem))
    tab_hardware.add_row("")
    tab_hardware.add_row("[bold white]Armazenamento (Disco /)[/bold white]")
    tab_hardware.add_row(desenhar_barra(dados["DISCO_ROOT_PERC"], f"[bold {cor_disco}]
{dados['DISCO_ROOT_PERC']}%[/]", cor_disco))

    painel_hardware = Panel(tab_hardware, title="⚙️ Recursos do Sistema",
border_style="cyan")

    # --- COLUNA DA DIREITA: REDE E DETALHES ---
    tab_rede = Table(show_header=False, expand=True, box=None)
    tab_rede.add_row(f"[bold cyan]Host:[/bold cyan] [bold magenta]{dados['HOSTNAME']}[/bold
magenta] | [bold cyan]IP:[/bold cyan] {dados['IP']}")
    tab_rede.add_row(f"[bold cyan]Memória Total Disponível:[/bold cyan]
{dados['TAMANHO_MEMORIA']}")
    tab_rede.add_row("")

    tab_rede.add_row("[bold green]↓ Velocidade de Download[/bold green]")
    tab_rede.add_row(desenhar_barra(dados["PERC_GRAFICO_IN"], f"[bold green]
{dados['REDE_IN_MBS']} MB/s[/]", "green"))
    tab_rede.add_row(f"[dim white]Total acumulado recebido: {dados['REDE_TOTAL_IN_MB']} MB[/
dim white]")
    tab_rede.add_row("")

    tab_rede.add_row("[bold orange3]↑ Velocidade de Upload[/bold orange3]")
    tab_rede.add_row(desenhar_barra(dados["PERC_GRAFICO_OUT"], f"[bold orange3]
{dados['REDE_OUT_MBS']} MB/s[/]", "orange3"))
    tab_rede.add_row(f"[dim white]Total acumulado enviado: {dados['REDE_TOTAL_OUT_MB']} MB[/
dim white]")

    painel_rede = Panel(tab_rede, title="🌐 Tráfego de Rede Real", border_style="magenta")

    corpo_dashboard = Columns([painel_hardware, painel_rede], expand=True)

    layout_final = Layout()
    layout_final.split_column(
        Layout(header_panel, size=3),
        Layout(corpo_dashboard)
    )
    return layout_final

# =====
# LAÇO PRINCIPAL DE ATUALIZAÇÃO
# =====
if __name__ == "__main__":
    os.system('cls' if os.name == 'nt' else 'clear')

    with Live(gerar_layout_dashboard(None), refresh_per_second=2, screen=True) as live:
        while True:
            if msvcrt.kbhit():
                tecla = msvcrt.getch()
                if tecla == b'\x1b':
                    break

            dados_atuais = coletar_dados_com_velocidade()

```

```
        live.update(gerar_layout_dashboard(dados_atuais))
        time.sleep(0.3)

print("\n[bold green]Painel encerrado com sucesso![/bold green]")
```